← Back to news (https://www.qiminfo.ch/en/news/)

QimTech

Clean Code by Benoît ()

1 December 2020

Clean Code: Definition and Principles

Everything you need to know about clean code: its definition, origin, fundamental principles, and how to implement it concretely.

Table of Contents

1. Clean Code by Benoît

2. Clean Code: Definition and Principles
   2.1. The Simple Definition of "Clean Code"
   2.2. Measuring code readability
   2.3. Clean code: The Book
   2.4. Clean code: The Principles
   2.5. Six steps to integrate clean code into your company
   2.6. Clean code applied to different languages
   2.7. The best tools for “clean coding”

Software development refers to the process of creating, designing, deploying, and supporting software. Since the dawn of computing, technologies have evolved significantly, but one prerequisite remains: the quality of the code is essential. The concept of “clean code” serves as a guide for developers through the complexity of computing. In this article, we explore this philosophy of clean code, its principles, and its impact on various programming languages.

Together, let’s define clean code, return to its origins, and explore its concrete fundamentals. Qim info and the Custom Software Development department (https://www.qiminfo.ch/en/custom-software-and-development/) are at your disposal to discuss this topic with you.

The Simple Definition of "Clean Code"

Clean code is **a set of practices aimed at producing understandable, efficient, and intuitive code to handle**.

The developer’s job is to write code intelligible by machines. However,in the world of industry, being understood by machines is not enough: in the vast majority of projects we work on, the same source code is manipulated by several people. And that’s where things get complicated: being understood by humans is not as easy as being understood by machines.

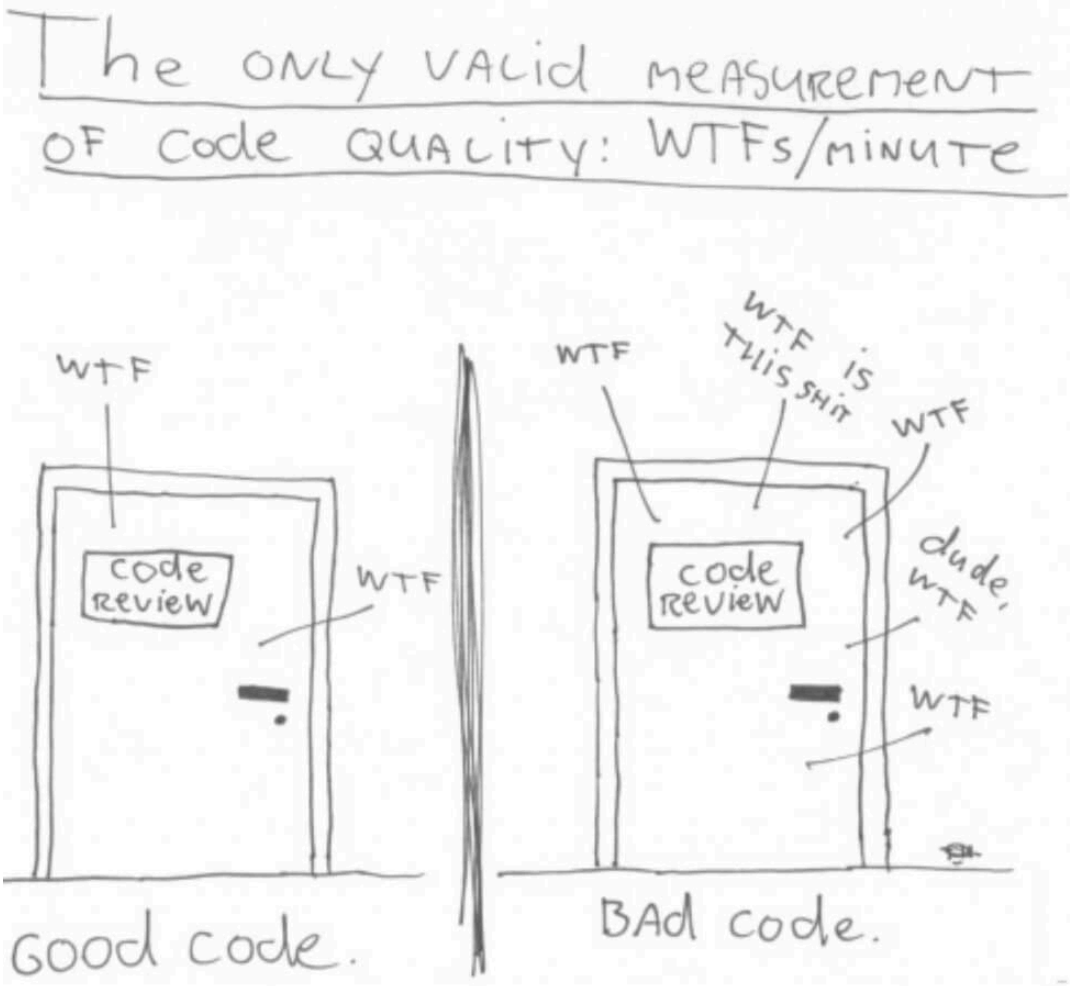
Indeed, for an application to evolve correctly, its code must be understood, adapted, and modified by any developer. If this is not the case, the cost of evolving the application will increase exponentially over time… This is where we can make a distinction between efficient code and clean code. Each of them focuses on different aspects of the programming process: while **clean code primarily aims to improve readability and maintainability**, efficient code focuses on performance and optimising program execution.

Moreover, how do we measure code readability?

Measuring code readability

How to determine if a code is clean? Or rather how clean a code is? We might think of Sonar, used with the Java language, which gives indications on the complexity, vulnerability, and test coverage of the written code.

However, a computerised tool will not really be able to determine how comprehensible the code will be to another developer, it will only give indications: the best indicator that proves it is the WTF/M, as seen below.



The only valid measurement of code quality: WTFs/minutes (source: osnews.com)

Beyond the meme, the illustration shows the goal sought by the clean coder: to reduce the rate of WTFs/Minutes, that is, to increase the readability of their code.

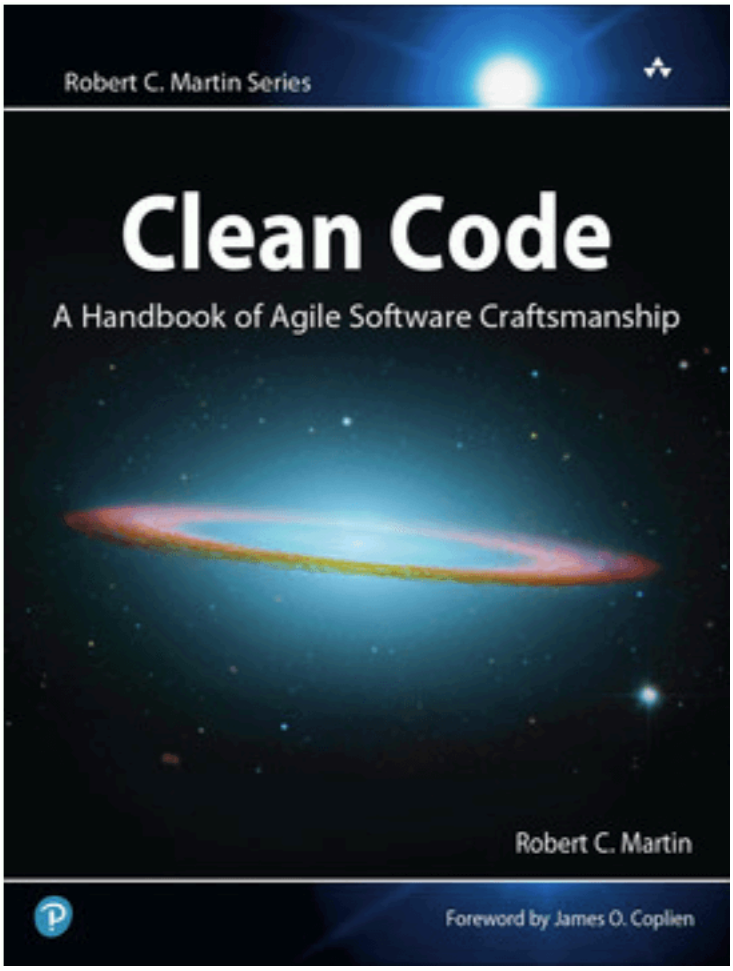
However, there are measures you can put in place to improve the readability of your code:

- Document and comment on your code, without polluting it with too many superfluous comments
- Stay consistent in your indentation style
- Reduce the nesting level
- Do not keep lines coded “just in case”

Thus, clean code will be readable by the largest number of developers, which will avoid misunderstandings later on. It’s the right moment to return to the origins of clean code.

Clean code: The Book

Luminaries of software development have tried to answer the question of establishing clean coding standards. One of the most famous, Robert C. Martin (aka Uncle Bob), has delved into the principles that define quality source code in his book *Clean Code: A Handbook of Agile Software Craftsmanship*, published in 2008. Indeed, this book is not recent, but it has inspired and continues to inspire generations of coders.



ISBN: 9780136083238

The principles developed in this book are general principles, agnostic of the programming language used. They are not rules to be strictly applied, but rather aids or inspirations.

Summary of Clean Code: A Handbook of Agile Software Craftsmanship

Robert C. Martin thus proposes a number of principles to implement to ensure that our code is clean:

- **Follow recognised conventions** to achieve the most universal code possible
- Leave the code cleaner than you found it, a principle also known under the expression **"the boy scout rule**"
- **Respect the fundamental design rules**, such as keeping configurable data at a high level or avoiding over-configuring functions
- **Promote code understanding**, by staying consistent, explicit, and facilitating the visibility of dependencies of different functions
- **Respect naming rules**: no ambiguity about the names of functions and variables, searchable names, and clear distinctions between entities
- Implement **short functions**, which have a single use, without side effects, and with as few arguments as possible
- **Take care with comments**, being clear, concise, avoiding redundancy and unnecessary noise
- **Structure the source code** by particularly caring for indentations, declaring variables, and keeping the code structurally coherent, for example by hiding internal structures for more readability
- Carry out **rapid tests**, independent, useful, repeatable, and easy to launch.

Robert C. Martin also gives us thesix **indicators of insufficiently clean code,**which he calls "code smells", and which are therefore to be monitored and avoided. Thus, code smells are:

- **Rigid** and therefore difficult to evolve
- **Fragile**, presents many dysfunctions
- **Immobile**, that is, it is difficult or even impossible to reuse for other projects
- Unnecessarily **complex**
- **Repetitive**
- **Opaque**, difficult to understand.

The SOLID Principles

It's a subset of the principles promoted by Robert C. Martin, referring to five basic principles for object-oriented programming:

- **Single Responsibility Principle:**each component has only one and unique responsibility.
- **Open Closed Principle**, which indicates that each component should be open to extension and closed to modification.
- **Liskov Substitution Principle**, which specifies that each subclass should be able to be substituted for its superclass without consequence.
- **Interface Segregation Principle**: no object should depend on methods it does not use.
- **Dependency Inversion Principle**: high-level components should not depend on low-level components.

Clean code: The Principles

Ultimately, what to remember from all these principles? How to establish standards for clean code? We choose to keep three, absolutely crucial:

The code must be simple

It's the first characteristic of clean code. As much as possible, one must avoid complicating the code by asking if the implemented solution is the simplest to meet the need. The simpler the code, the more readable it is, and therefore it will be more maintainable in the future. There is an acronym that helps us remember this principle: KISS (Keep It Simple Stupid).
For this**, it is advisable to avoid over-configurability and to prohibit everything that is written "just in case".**

The code must be focused

A piece of code must be written to solve a single and unique problem. This is valid at all levels of code abstraction: method, class, package, or module. The thought process around the division of code should allow determining the right abstractions.

> *"Bad abstraction is worse than duplication"*
>
> Two pieces of code can be identical: if they each represent different concepts, it is preferable to keep this "accidental" duplication. In this case, the DRY (Don't Repeat Yourself) principle does not apply.

Thus, the responsibilities of each piece of code will be correctly distributed. Here we find the SOLID principles, notably the "Single Responsibility Principle".

The code must be testable

If the code is simple and has only one responsibility, it should be easy to test. Tests represent a safety against regressions that may occur during the evolution of the code. They constitute also the documentation that allows other developers to understand the operation of the tested code. They should preferably be written before the productive code (TDD) and be automated as much as possible.
We can present the FIRST acronym that characterises the "clean" tests: Fast, Independent, Repeatable, Self-Validating, Timely.

Now let's move on to a concrete method to integrate clean code into your company.

Six steps to integrate clean code into your company

If you wish to introduce clean code methods into your company's operations, it may be good to call on experts in the field. Qim info will have the consultant who will meet your particular need.

Here is a set of recommendations to optimise your software productions through clean code:

1. **Remind developers of the principles of clean code** so that all your teams are on the same wavelength. This is also the opportune moment to consider training or a seminar on the subject.
2. **Implement a clean code policy** by clearly defining the most important priority rules within the scope of your activity. For example, by standardising the names of variables or functions you use regularly.
3. **Document** the code produced by your teams so that it remains readable over time.
4. **Integrate a Linter,**a utility that will allow you to maintain a uniform code based on rules you have configured and predefined. There are several for different coding languages.
5. **Organise monitoring**, asking your developers to regularly review their own code to ensure its cleanliness, and organising team debriefings.
6. **Rework old codes**. Just because they weren't clean before doesn't mean they shouldn't follow your transition!

Clean code applied to different languages

The rules of clean code remain the same across all computing languages, but here are some of the particularities specific to each language.

Python

The most important will be the naming of functions and variables, as well as avoiding double negatives.

Java

While Java does not propose a default project structure, it would however be interesting to follow a structure you have predefined and which will remain constant.

Javascript

The naming of different elements is most important for you to develop clean code on Javascript.

C#

It is particularly important to pay attention to duplicates in code on C#.

html

Redundant and double tags are your main enemies on html for clean code.

You may also be interested in these articles...

See all news (https://www.qiminfo.ch/en/news/)

(https://www.qiminfo.ch/en/power-bi-pour-powerapps/) QimTech

(https://www.qiminfo.ch/en/project-management/) QimTech

(https://www QimTech

Specifica to the IT

Ruby on rails

The name of variables is once again paramount. We particularly recommend keeping the same vocabulary for similar variables.

There are also useful tools for clean code, some of which are specific to one language or another.

The best tools for “clean coding”

There is no tool that can implant clean code or automatically clean up your productions, however, some tools can facilitate the task and give you indicators to follow.

- **Linters** as we previously mentioned, these are utilities that allow you to predefine rules that your code will have to respect. There are dozens for each language: checkstyle for Java, cppcheck for C++, pylint for Python, etc.
- **Maven** is a code structure tool for Java
- **Rubocop** analyses and formats your code on Ruby on Rails
- **Prettier** automates your code formatting
- **SonarQube** analyses code during the compilation phase and is used with the most commonly used programming languages in software development (Python, C#, Java…).
- **Jenkins (or another continuous integration tool)**: continuous integration and automated testing help maintain code quality throughout the development cycle.
- **JUnit (or any other unit testing framework):** writing unit tests is a key component of clean code. JUnit is a popular framework for unit tests in Java.

Clean code methods are thus guides that allow you to make your code simpler to use, more readable, and more durable. So, what are you waiting for to also get started with clean code? Our experts will know how to listen to you and advise you in your approach. To learn more, we invite you to consult our article on the stages of custom software development. (https://www.qiminfo.ch/en/6-simple-steps-to-custom-software-development/)

Benoît.

How do I integrate a PowerApps application into a Power BI dashboard? (https://www.qiminfo.ch/en/power-bi-pour-powerapps/)

Discover how to connect your business applications to your dashboards to interact with your data in real time and automate your processes. […]

Read more (https://www.qiminfo.ch/en/power-bi-pour-powerapps/)

Project Management: definition, methods and key tools (https://www.qiminfo.ch/en/project-management/)

Project management structures every stage, from defining objectives to closing the project. Discover its methods, tools and certifications to guarantee successful projects. […]

Read more (https://www.qiminfo.ch/en/project-management/)

(https://www.qiminfo.ch/en/power-des-cl

Learn how specificati and tips fo […]

(https://

© Qim info – Design : WeAreBandits (https://www.wearebandits.love/) – Growth : Nexoka (https://nexoka.ch/?utm_source=webclient&utm_medium=web&utm_campaign=web_qiminfo)